

Cloud Image Registry & Distribution - Lab Report

Name: 唐雯婷__

Student ID: 202383910018__

Class: 23 人工智能__

Date: 2026.5.28__

1. Experimental Purpose

- Learn how to use Docker Tags to manage different versions of cloud applications.
- Understand cloud image distribution using GitHub Container Registry (GHCR).
- Practice secure authentication using GitHub Personal Access Tokens (PAT).
- Understand Docker layered storage and disaster recovery deployment.
- Compare standard Docker images with Alpine images and analyze optimization benefits.

2. Experimental Environment

- Operating System: Windows
- Docker Desktop / Docker Engine
- GitHub Account
- GitHub Codespaces
- Git
- GitHub Container Registry (GHCR)

3. Experimental procedures

Step 1: Secure Authentication (PAT Generation)

- Open GitHub Settings → Developer settings → Personal access tokens → Tokens (classic).
- Generate a new token and enable write:packages permission.
- Copy and save the generated token.
- Run the login commands in Codespaces terminal.

Command Used

```
export CR_PAT=YOUR_COPIED_TOKEN
```

```
echo $CR_PAT | docker login ghcr.io -u YOUR_GITHUB_USERNAME --password-stdin
```

Result and Analysis

Describe the execution result here, including whether the command succeeded, what output appeared, and what was learned from this step.

Screenshot Placeholder

Settings / Developer Settings

scopes you've selected are included in other scopes. Only the minimum set of necessary scopes has been saved.

Personal access tokens (classic)

Generate new token

Tokens you have generated that can be used to access the [GitHub API](#).

Make sure to copy your personal access token now. You won't be able to see it again!

ghp_e2NTdYMiypwCuY61PCY48EEz2Imq3I2TsFUz

Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

```
问题 输出 调试控制台 终端 端口
To explore VS Code to its fullest, search using the Command Palette (Cmd/Ctrl + Shift + P or F1).
Edit away, run your app as usual, and we'll automatically make it available for you to access.
@WentingTang →/workspaces/cloud-computing-lab-github (main) $ export CR_PAT=ghp_e2NTdYMiypwCuY61PCY48EEz2Imq3I2TsFUz
@WentingTang →/workspaces/cloud-computing-lab-github (main) $ echo $CR_PAT | docker login ghcr.io -u WentingTang --password-stdin

WARNING! Your credentials are stored unencrypted in '/home/codespace/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
@WentingTang →/workspaces/cloud-computing-lab-github (main) $
```

Step 2: Build and Tag Docker Image

- Build the Docker image named cloud-lab-image:v1.
- Tag the image into cloud registry format.

Command Used

```
docker build -t cloud-lab-image:v1 .
```

```
docker tag cloud-lab-image:v1 ghcr.io/YOUR_USERNAME/cloud-app:v1.0
```

Result and Analysis

Describe the execution result here, including whether the command succeeded, what output appeared, and what was learned from this step.

Screenshot Placeholder



The first screenshot shows the output of the `docker build -t cloud-lab-image:v1 .` command. It displays the extraction of the image, copying of files, and the final naming of the image to `docker.io/library/cloud-lab-image:v1`.

The second screenshot shows the output of the `docker images` command, displaying a table of images:

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cloud-lab-image:v1	e24619d8ba20	238MB	63.1MB	

Step 3: Push Image to GitHub Container Registry

- Push the image to GHCR.
- Verify the uploaded package in GitHub Packages.

Command Used

```
docker push ghcr.io/YOUR_USERNAME/cloud-app:v1.0
```

Result and Analysis

Describe the execution result here, including whether the command succeeded, what output appeared, and what was learned from this step.

Screenshot Placeholder

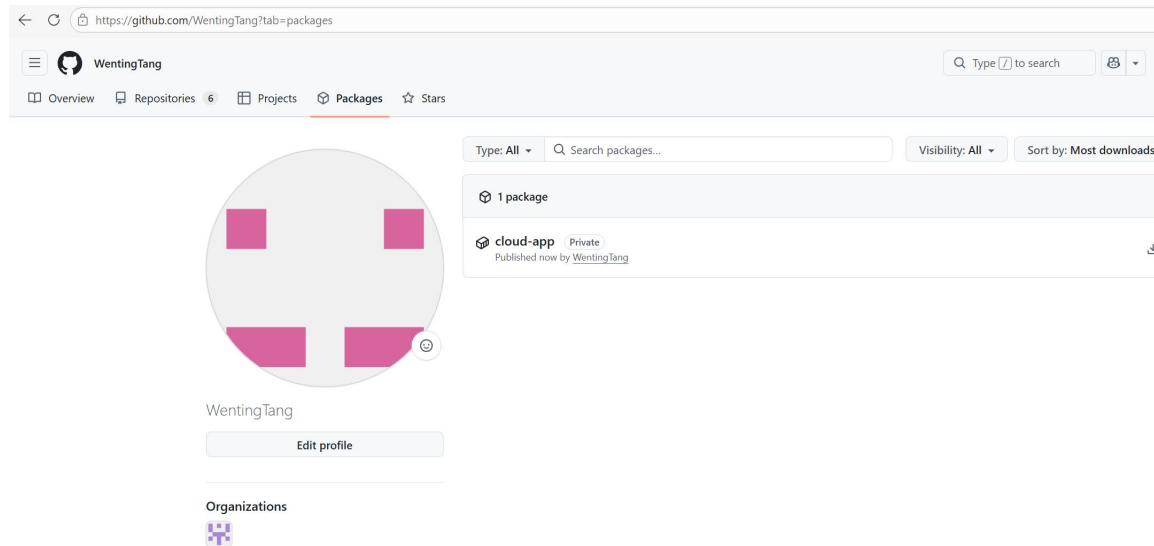
```
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker tag cloud-lab-image:v1 ghcr.io/wentingtang/cloud-app:v1.0
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
cloud-lab-image:v1	e24619d8ba20	238MB	63.1MB	
ghcr.io/wentingtang/cloud-app:v1.0	e24619d8ba20	238MB	63.1MB	

```
@WentingTang → /workspaces/cloud-computing-lab-github (main) $
```



```
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker push ghcr.io/wentingtang/cloud-app:v1.0
The push refers to repository [ghcr.io/wentingtang/cloud-app]
7f8b1a2b17d8: Pushed
830625e1ac85: Pushed
5431d0092ffd: Pushed
13fd728be9eb: Pushed
2d6df098680a: Pushed
45381ecb0e2f: Pushed
43903c8bd62c: Pushed
b4a248c845e5: Pushed
5b4d6ff92fc4: Pushed
v1.0: digest: sha256:e24619d8ba20bcd4fd5bc54cc541a45617dd4a524cf22b1e64f44eae3377aefc size: 856
@WentingTang → /workspaces/cloud-computing-lab-github (main) $
```



Step 4: Iteration and Layer Analysis

- Modify the HTML content to create Version 2.0.
- Rebuild and retag the new image.
- Push the new image and observe the Layer already exists message.

Command Used

```
docker build -t cloud-lab-image:v2 .
```

```
docker tag cloud-lab-image:v2 ghcr.io/YOUR_USERNAME/cloud-app:v2.0
```

```
docker push ghcr.io/YOUR_USERNAME/cloud-app:v2.0
```

Result and Analysis

Describe the execution result here, including whether the command succeeded, what output appeared, and what was learned from this step.

Screenshot Placeholder

```
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v2 .
=> => transferring context: 6.40kB
=> CACHED [1/2] FROM docker.io/library/nginx:latest@sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
=> => resolve docker.io/library/nginx:latest@sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
=> [2/2] COPY . /usr/share/nginx/html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:d4b316ad92ebf8e2f38ea8b0ea64a7857ef482199d9b585457b0add645f31ef5
=> => exporting config sha256:1f006abb729a34e55c1377062054311684feac8b06226fcb2d047d9ece4d1379
=> => exporting attestation manifest sha256:2d0c90164cb6333569183601a93dd1213f93b6b3d5d3669f4397d86cc0c96808
=> => exporting manifest list sha256:225fb858dfab0b42d9092df4b94c097c310b0d9008183e96e8899757d6a1ad63
=> => naming to docker.io/library/cloud-lab-image:v2
=> => unpacking to docker.io/library/cloud-lab-image:v2
```

```
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker push ghcr.io/wentingtang/cloud-app:v2.0
The push refers to repository [ghcr.io/wentingtang/cloud-app]
5431d0092ffd: Layer already exists
5b4d6ff92fc4: Layer already exists
7f8b1a2b17d8: Layer already exists
13363500f0b5: Pushed
f4a2a3ce8447: Pushed
830625e1ac85: Layer already exists
45381ecb0e2f: Layer already exists
b4a248c845e5: Layer already exists
13fd728be9eb: Layer already exists
v2.0: digest: sha256:225fb858dfab0b42d9092df4b94c097c310b0d9008183e96e8899757d6a1ad63 size: 856
@WentingTang → /workspaces/cloud-computing-lab-github (main) $
```

Step 5: Disaster Recovery (Pull Test)

- Delete all local Docker data to simulate a new server.
- Pull and run the cloud image directly from GHCR.
- Verify the deployment using browser access.
- Solve the port conflict problem when running another container.

Command Used

```
docker system prune -a
```

```
docker run -d -p 8888:80 ghcr.io/YOUR_USERNAME/cloud-app:v1.0
```

```
docker run -d -p 8888:80 ghcr.io/YOUR_USERNAME/cloud-app:v2.0
```

Result and Analysis

Describe the execution result here, including whether the command succeeded, what output appeared, and what was learned from this step.

Screenshot Placeholder

```
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker system prune -a
deleted: sha256:2d0c90164cb6333569183601a93dd1213f93b6b3d5d3669f4397d86cc0c96808

Deleted build cache objects:
t2sixwqkcw5exdrusti2thdf
lytxqgzhticbo9tjmp8ldbtvw
xo6nsinfvrgnidnb8sey6c9h4
y2s7m0v8qghwyzu5tpdrbdz5x
46mm5i5klffhw6mkmj0dozi9t
z163wdpzxt21muqc138r8nu7u
r0roq9x2oaty0uflrhgtbg6wk
hzqxbmr7q8i7hsm6obx237uts
qd9bzhppqxphk5f5n1awai0a3

@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/wentingtang/cloud-app:v1.0
5431d0092fffd: Pull complete
830625e1ac85: Pull complete
2d6df098680a: Download complete
Digest: sha256:e24619d8ba20bcd4fd5bc54cc541a45617dd4a524cf22b1e64f44ae3377aecf
Status: Downloaded newer image for ghcr.io/wentingtang/cloud-app:v1.0
b39c8a37c18aff9fd73b8bc5abbb79d9c7436d21ddb106ebef29ff65ed3a0270
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
b39c8a37c18a   ghcr.io/wentingtang/cloud-app:v1.0   "/docker-entrypoint..." 13 seconds ago Up 10 seconds  0.0.0.0:8888->80/tcp, [
88->80/tcp     tender_faraday
@WentingTang → /workspaces/cloud-computing-lab-github (main) $
```

Cloud-Native Deployment Success!

This site was deployed using **GitHub Actions**.

• [Site Status: Live](#)

Student Name: WentingTang

Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)



```
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/wentingtang/cloud-app:v2.0
Unable to find image 'ghcr.io/wentingtang/cloud-app:v2.0' locally
v2.0: Pulling from wentingtang/cloud-app
f4a2a3ce8447: Pull complete
13363500f0b5: Download complete
Digest: sha256:225fb858dfab0b42d9092df4b94c097c310b0d9008183e96e8899757d6a1ad63
Status: Downloaded newer image for ghcr.io/wentingtang/cloud-app:v2.0
7acbe0091e39c8f7385e30cafd2b179c921b380bf441b17ef8986736089435dc
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint insp
iring_morse (f5a7559202170299db76c7b44df833c153b1319ef4aee4d0455bc3b084d9a1d7): Bind for ::8888 failed: port is already allocated

Run 'docker run --help' for more information
@WentingTang → /workspaces/cloud-computing-lab-github (main) $

@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker ps
b39c8a37c18a   ghcr.io/wentingtang/cloud-app:v1.0   "/docker-entrypoint..." 5 minutes ago   Up 5 minutes   0.0.0.0:8888->80/tcp, [::]:8888
->80/tcp     tender_faraday
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker stop b39c8a37c18a
b39c8a37c18a
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/wentingtang/cloud-app:v2.0
9f747dd0482ac1c2697afcc838f86f2befddea2a1c99c0e3b0ed9fb8cee78a35
@WentingTang → /workspaces/cloud-computing-lab-github (main) $ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
9f747dd0482a   ghcr.io/wentingtang/cloud-app:v2.0   "/docker-entrypoint..." 12 seconds ago Up 12 seconds  0.0.0.0:8888->80/tcp, [::]:88
88->80/tcp     festive_nobel
@WentingTang → /workspaces/cloud-computing-lab-github (main) $
```


Step 6: Image Optimization (Standard vs Alpine)

- Create a lightweight Alpine Dockerfile.
- Build both standard and Alpine versions.
- Compare image sizes and analyze differences.

Command Used

```
FROM nginx:alpine

COPY . /usr/share/nginx/html

docker build -t cloud-app:standard -f Dockerfile .

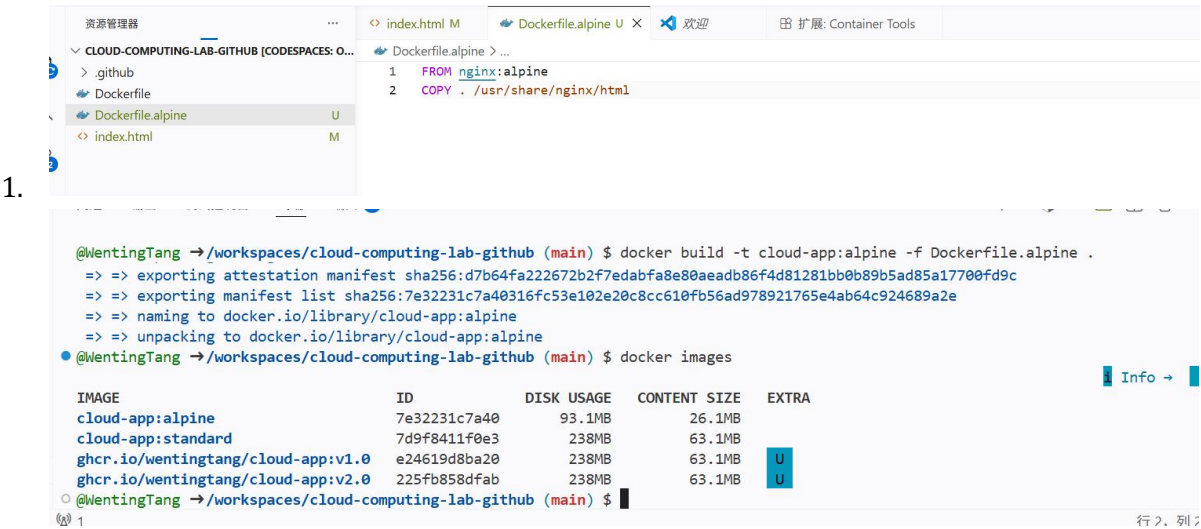
docker build -t cloud-app:alpine -f Dockerfile.alpine .

docker images | grep nginx
```

Result and Analysis

Describe the execution result here, including whether the command succeeded, what output appeared, and what was learned from this step.

Screenshot Placeholder



4. Conclusion and reflection

Through this experiment, I successfully learned how to build, tag, push, pull, and manage Docker images using GitHub Container Registry (GHCR). I gained practical experience with secure authentication through Personal Access Tokens (PATs), image version control using Docker tags, and cloud-based image distribution. During the experiment, I encountered challenges such as registry authentication and port conflicts during container deployment, but these issues were resolved through proper configuration and container management. I also observed Docker's layer

caching mechanism, which significantly improved the efficiency of pushing updated images by uploading only the modified layers. Furthermore, by comparing standard and Alpine-based images, I understood the importance of image optimization in reducing storage costs, network usage, and deployment time. Overall, this lab enhanced my understanding of cloud-native deployment, container technologies, DevOps practices, and modern software delivery workflows, providing valuable hands-on experience for future cloud computing projects.